

Module 03

Linux CLI Fundamentals



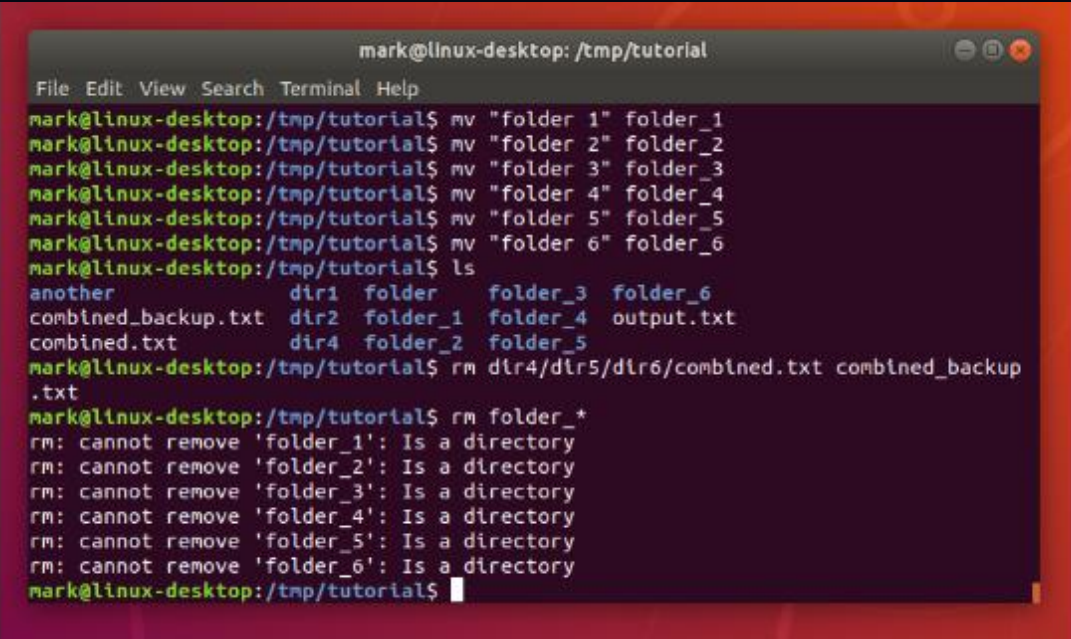


01. Introduction to Shell & CLI

Understanding user space interfaces, REPL operations, and system interactions

What is an Operating System Shell?

- **Command Interpreter:** A shell is an interface program that reads user-typed commands and translates them into Kernel operations.
- **User-Space Layer:** It executes strictly in user-space, shielding the core kernel from direct, unauthorized application space contact.
- **Automative Environment:** Houses programming features (variables, control loops, functions) to build shell scripts for infrastructure automation.



```
mark@linux-desktop: /tmp/tutorial
File Edit View Search Terminal Help
mark@linux-desktop:/tmp/tutorial$ mv "folder 1" folder_1
mark@linux-desktop:/tmp/tutorial$ mv "folder 2" folder_2
mark@linux-desktop:/tmp/tutorial$ mv "folder 3" folder_3
mark@linux-desktop:/tmp/tutorial$ mv "folder 4" folder_4
mark@linux-desktop:/tmp/tutorial$ mv "folder 5" folder_5
mark@linux-desktop:/tmp/tutorial$ mv "folder 6" folder_6
mark@linux-desktop:/tmp/tutorial$ ls
another          dir1  folder  folder_3  folder_6
combined_backup.txt  dir2  folder_1 folder_4  output.txt
combined.txt      dir4  folder_2 folder_5
mark@linux-desktop:/tmp/tutorial$ rm dir4/dir5/dir6/combined.txt combined_backup
.txt
mark@linux-desktop:/tmp/tutorial$ rm folder_*
rm: cannot remove 'folder_1': Is a directory
rm: cannot remove 'folder_2': Is a directory
rm: cannot remove 'folder_3': Is a directory
rm: cannot remove 'folder_4': Is a directory
rm: cannot remove 'folder_5': Is a directory
rm: cannot remove 'folder_6': Is a directory
mark@linux-desktop:/tmp/tutorial$
```

| Why Do We Adopt the Term "Shell"?

The Outer Structural Metaphor

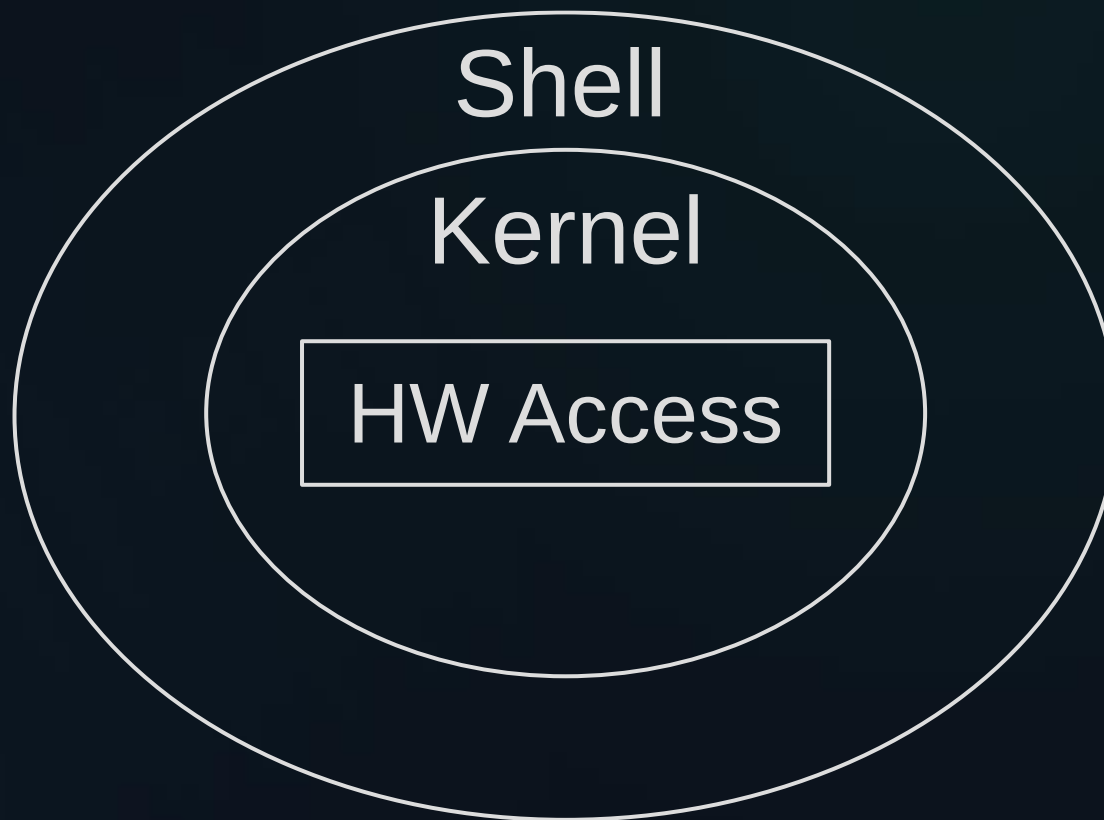
In biology, a shell protects and encapsulates the vital organic matter inside. In computer systems engineering:

- The **Kernel** is the sensitive organic core (handles CPU scheduling, RAM allocations, drivers).
- The **Shell** is the hard, user-facing surface enclosing the inner system.

Historical Roots & Origin

The term was established in 1964 by computer scientist Louis Pouzin for the Multics Operating System:

- Pouzin conceptualized running command parameters like scripts.
- He called it a "shell" to signify an easily swap-ready command handler encasing system services.



Command-Line Interface: OPERATING LIKE A REPL

The CLI Environment

The CLI is a text-centric interface prioritized by engineers for raw scriptable control and a low system memory footprint.

It behaves explicitly as a continuous interpreter loop to manage system standard I/O streams directly.

The REPL Lifecycle

The environment parses commands sequentially:

- 1. Read:** Captures text from standard input.
- 2. Eval:** Parses variables and executes commands.
- 3. Print:** Outputs results to stdout/stderr.
- 4. Loop:** Resets to prompt, waiting for input.

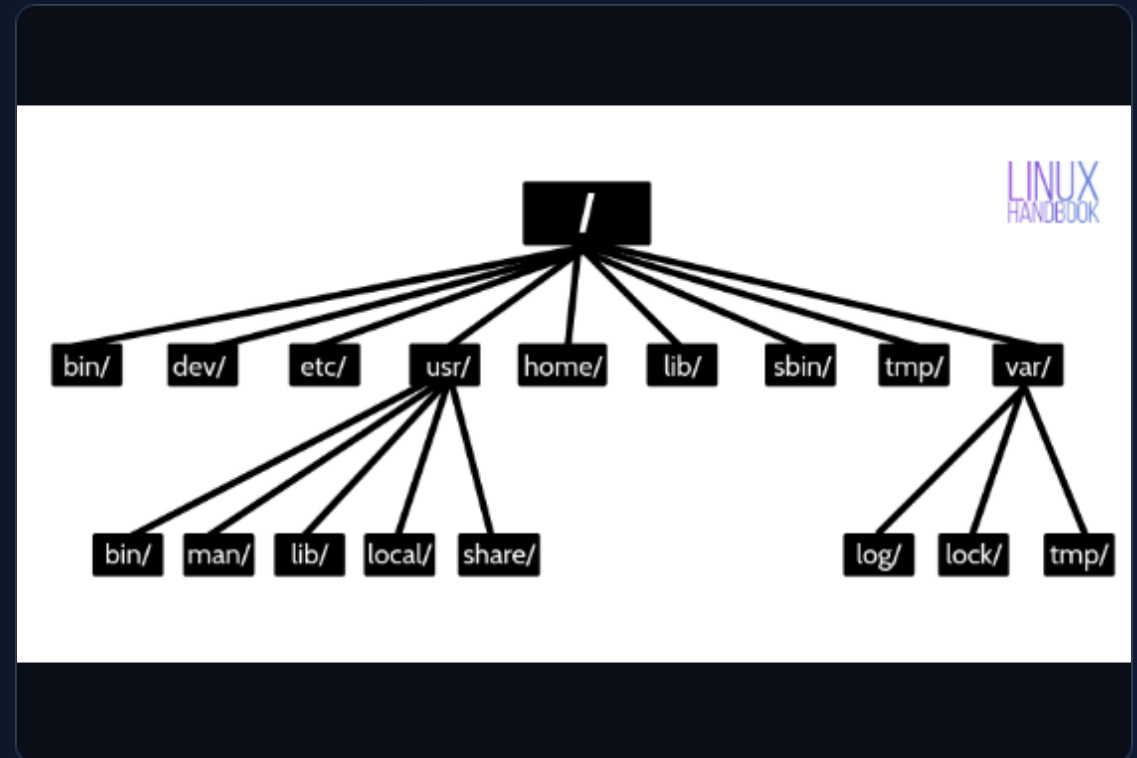


02. Filesystem Hierarchy Standard

Navigating the standard structural design of a Linux environment

| The Filesystem Hierarchy Standard (FHS)

- **The Unified Tree:** Unlike Windows, which maps directories to device letters (C:, D:), Linux structures all drives and partitions under one root tree node (/).
- **Standardized Mapping:** The **FHS** is maintained by the Linux Foundation to guarantee identical binary and file layouts across different distributions.
- **Portability Assurance:** This standard ensures that third-party software installs safely without disrupting vital system core operations.



Common Paths in FHS & Purposes

Path	Category Name	Standard Purpose & Ubuntu Context
/etc	Host-specific Configs	System-wide startup files and networking rules. Contains no executable code.
/var	Variable Data files	Spool directory queues, system access logs (/var/log), and runtime states.
/usr	User System Resources	Contains the secondary hierarchy for multi-user read-only system utilities.
/home	User Home folders	Personal storage directories for standard users containing settings and keys.
/tmp	Temporary Storage	Holds volatile program states. Contents are cleared upon system restart.
/run	Ephemeral Runtime State	Tracks runtime data since last boot (e.g., active process ID files, sockets).

Command: `pwd`

Print Working Directory

- Core Mechanics: Emits the exact absolute path starting from / to your current terminal position.
- No Variables Needed: Directly accesses shell environment variables tracking current process states.
- Options: -P resolves physical symlinks to show real, actual directories instead of symbolic names.

Example 1: Basic execution in home directory

```
ubuntu@server:~$ pwd  
/home/ubuntu
```

Example 2: Checking location within symlinked paths

```
ubuntu@server:/bin$ pwd -P  
/usr/bin
```

Command: `ls`

List Directory Contents

- Content Inspection: Displays information about system files and subdirectory catalogs.
- Key Flag Options:
 - -l: Long-listing details (permissions, sizes, dates).
 - -a: Includes hidden files beginning with dot.
 - -h: Renders human-readable bytes (K, M, G).

Example 1: Basic listing of active directory

```
ubuntu@server:~$ ls
development  projects  readme.txt
```

Example 2: Detailed inspection showing hidden files

```
ubuntu@server:~$ ls -lah
total 16K
drwxr-x--- 4 ubuntu ubuntu 4.0K Jul 14 17:25 .
drwxr-xr-x 3 root   root   4.0K Jul 14 17:24 ..
-rw-r--r-- 1 ubuntu ubuntu 220  Jul 14 17:24 .bash_logout
drwxrwxr-x 2 ubuntu ubuntu 4.0K Jul 14 17:25 projects
```

Command: `cd`

Change Directory

- Directory Navigation: Modifies the active terminal's position to a newly target path.
- Smart Shortcuts:
 - `cd` or `cd ~`: Jumps to current home.
 - `cd -`: Restores previous directory state.
 - `cd ..`: Escapes to parent directory level.

Example 1: Switch path with absolute target

```
ubuntu@server:~$ cd /var/log/nginx/
```

```
ubuntu@server:/var/log/nginx$
```

Example 2: Return to user home using shortcut

```
ubuntu@server:/var/log/nginx$ cd ~
```

```
ubuntu@server:~$
```

Absolute vs. Relative Paths

Absolute Paths

Defines the complete, unambiguous location of a resource relative to the system's root directory (`/`). Always starts with a slash.

- Root Anchor: Starts from `/`
- Independent: Identical resolve regardless of the active shell's current working directory state.
- Format: `/var/log/nginx/access.log`

Relative Paths

Specifies resource locations relative to the shell session's active directory. Relies on state-based dot pointers.

- State Dependent: Resolves from current directory.
- Pointers: Uses `.` (current dir) and `..` (parent dir).
- Format: `../nginx/access.log`

Command: `cat`

Concatenate and Display Files

- Content Dumping: Outputs raw file bytes directly onto standard output stdout stream.
- Concatenation: Chains multiple text inputs sequentially, merging results.
- Usage Limit: Not advised to use for huge logfiles, since terminal processes can block memory. (Use less instead).

Example 1: Read server configuration file

```
ubuntu@server:~$ cat /etc/hostname  
production-ubuntu-server
```

Example 2: Combine file arrays to output target

```
ubuntu@server:~$ cat part1.txt part2.txt > full.txt
```

Command: `touch`

Create Empty Files / Update Timestamps

- File Initialization: Easily initializes new, completely blank files without feeding payload content.
- Metadata Modification: Modifies the file's latest Access Time (atime) and Modification Time (mtime).
- Common Use: Triggers build jobs or automated script updates waiting on file timestamp transitions.

```
# Example 1: Initialize multiple config files  
ubuntu@server:~$ touch index.js variables.env  
  
# Example 2: Modify existing file timestamp  
ubuntu@server:~$ touch -m index.js  
# mtime updated to system's current datetime
```

Commands: `mkdir` & `rmdir`

Directory Provisioning

- Creation: mkdir generates brand new folders.
- Parent Propagation: Option -p creates parent hierarchy structures nested on-the-fly without complaining if parent levels do not exist.
- Destruction (rmdir): Destroys directory containers, but only if they are ****entirely empty**** (failsafe security protection).

Example 1: Creating nested directory path

```
ubuntu@server:~$ mkdir -p app/src/controllers
```

Example 2: Cleaning up empty directory paths

```
ubuntu@server:~$ rmdir -p app/src/controllers
```

Succeeds since all directories inside are empty

Command: `cp`

Copy Files and Directories

- Duplicate Files: Copies file payloads from source path targets over to new location tracks.
- Recursive Folders: Must apply flag -r or -R to force duplication on subdirectories.
- Preservation: Flag -a archives properties, preserving file ownership tags and Unix permissions.

Example 1: Duplicate configuration file

```
ubuntu@server:~$ cp app.conf app.conf.backup
```

Example 2: Copy whole directories recursively

```
ubuntu@server:~$ cp -r /var/www/html /tmp/backup_html
```

Command: `mv`

Move or Rename Files

- Dual Function: Relocates items, or alters file naming labels in-place.
- Instant Execution: When moving files on the exact same physical partition, only inode directory mapping tables update. No data copy occurs.
- Failsafe Mode: Parameter `-i` prompts users for manual confirmation before overwriting targets.

Example 1: Rename script inside directory

```
ubuntu@server:~$ mv main.js app.js
```

Example 2: Relocate file to system bin path

```
ubuntu@server:~$ mv backup.sh /usr/local/bin/
```

Command: `rm`

Remove Files and Directories

- Destructive Command: Linux command-line deletions are instantaneous and ****permanent****. There is no "trash can".
- Recursive Force:
 - -r: Recursively delete folders.
 - -f: Force mode, ignoring missing items and avoiding prompt warnings.

```
# Example 1: Clean up a temp log file
```

```
ubuntu@server:~$ rm session.log
```

```
# Example 2: Destructive folder deletion (Caution)
```

```
ubuntu@server:~$ rm -rf /tmp/staging_zone
```

SYSTEM NAVIGATION COMMANDS

`pwd` (Print Working Directory)

Outputs your current absolute path locator.

`cd` (Change Directory)

Swaps directory context. A raw ``cd`` returns you to `~``.

`ls` (List Contents)

Displays folder items. Use flags to details formatting.

```
$ pwd
/home/ubuntu
$ ls -lah /var/log/
drwxr-xr-x 13 root syslog 4.0K
-rw-r----- 1 root adm 240K syslog
```

Ubuntu Terminal

FOLDER & FILE CONSTRUCTION



mkdir

Creates system folders.

```
mkdir -p lab/q1
```

Ubuntu Terminal



rmdir

Removes empty folders.

```
rmdir lab/q1
```

Ubuntu Terminal



touch

Generates blank files.

```
touch main.cpp
```

Ubuntu Terminal

ESSENTIAL FILE OPERATIONS

Command	Standard Execution Syntax & Flags	Practical Example on Ubuntu Server
cp (Copy)	<code>`cp`</code> (use <code>`-r`</code> to copy folder nodes)	<code>cp -r /etc/nginx/ ~/nginx_backup/</code>
mv (Move)	<code>`mv`</code> (remaps index structures)	<code>mv config.tmp /etc/app/config.conf</code>
rm (Remove)	<code>`rm`</code> (use <code>`-rf`</code> to clear directory lists)	<code>rm -rf ~/temp_folder</code>
cat (Concatenate)	<code>`cat`</code> (dumps raw content to screen stdout)	<code>cat /etc/resolv.conf</code>

⚠ Critical Warning: ``rm -rf`` deletes paths recursively and completely bypasses trash prompts. Double-check all targets before execution.

03. Identity & Structure

Understanding users, groups, UIDs,
and GIDs in the Linux operating system.

What is a **User & UID**?

The User Concept

On Linux, a user represents any entity that can interact with the system, execute files, and own resources. This ensures complete isolation and customized access control.

Users are not restricted to real humans; background tasks, servers, and automated services also execute as isolated system users for secure operations.

User ID (UID)

The kernel ignores usernames entirely, referencing entities via numeric **User IDs (UIDs)**.

- # **UID 0:** Superuser/Root Account
- # **UID 1-999:** System Services (System Users)
- # **UID 1000+:** Interactive/Normal Users

What is a Group & GID?

The Group Concept

A group is a logical collection of users designed to simplify system-wide privilege management. Rather than assigning access permissions user-by-user, you assign them to a group.

In Ubuntu, users always have a **Primary Group** (created automatically with their name) and can join multiple **Secondary Groups** for extra system access.

Group ID (GID)

Similar to UIDs, Linux tracks groups internally through unique numeric **Group IDs (GIDs)**.

- # GID 0: Root/Superuser Group
- # GID 1-999: System Service Groups
- # GID 1000+: Regular User Groups

Normal Users vs Superuser (root)

Characteristic	Superuser (root)	Normal Users
User Identifier (UID)	Strictly 0	Usually ranges from 1000 upwards to 65534
Access Privileges	Unrestricted. Bypasses all discretionary file permission checks.	Restricted to own home directory and public system files.
Default Prompt Symbol	# (Indicates active high-risk administrative shell)	\$ (Indicates safe, regular user privileges)
Default Home Directory	/root	/home/
System Modification	Can modify kernel parameters, package updates, and configurations.	Requires sudo command authorization to execute admin tasks.

How to Read `/etc/passwd`

The `/etc/passwd` file is a plain-text database storing local user account parameters. Every entry is colon-separated:

```
ubuntu:x:1001:1001:John Doe,:,:/home/ubuntu:/bin/bash
```

1. Username

Name used to authenticate.

2. Password

'x' means hash is secure in `/etc/shadow`.

3. UID

Unique numeric user identifier.

4. GID

Primary group ID number.

5. GECOS (Info)

Full name, room details, contact.

6. Home Dir

User's starting absolute path.

7. Shell

Default command interpreter launched upon login.

How to Read `/etc/group`

The `/etc/group` file catalogs all definitions and user memberships of groups across the local system.

```
sudo x 27 ubuntu,developer,admin
```

1. Group Name

Label assigned to the GID.

2. Password Placeholder

Typically 'x', actual hash is in secure `/etc/gshadow`.

3. GID

Unique numeric group identifier.

4. User List

Comma-separated list of secondary group members.

04. Identity Commands

Interactive tools and management procedures for user and group administration on Ubuntu.

The **id** Command

Queries and displays actual active identification numbers (UID, GID) and group associations for validating privilege mappings.



Check Current

Outputs numeric values and parenthesized string labels of the logged-in user.

```
$ id
```



Inspect Other

Queries database details for any alternate registered username on the server.

```
$ id administrator
```



Print Groups

Extracts only textual names of all secondary group memberships of the user.

```
$ id -nG
```

The **adduser** Command

A high-level Perl script on Ubuntu to easily create users with custom homes, shells, and passkey assignments interactively.

Standard Run

Standard interactive profile builder. Auto-configures home folder, permissions, and passwords.

```
$ sudo adduser john
```

Custom Home

Spins up a new user with a specified alternative home directory instead of standard pathing.

```
$ sudo adduser --home /data/john  
john
```

Direct Shell

Configures a distinct shell during setup (e.g. standard sh instead of default bash environment).

```
$ sudo adduser --shell /bin/sh  
john
```

The **deluser** Command

Safely removes users and associated environment folders, offering protection switches to back up crucial workspace data first.



Standard Delete

Deletes the user from database but leaves their physical home folder intact.

```
$ sudo deluser john
```



Purge Home

Fully eradicates user registration AND permanently deletes their entire home folder.

```
$ sudo deluser --remove-home  
john
```



Archive & Clean

Creates a gzip backup of the user's home folder in root before initiating file cleanups.

```
$ sudo deluser --backup john
```


The **passwd** Command

Handles user authentication settings, enabling passwords to be changed, expired, locked, or unlocked.



Self Update

Prompts standard user directly to change their existing authentication passcode securely.

```
$ passwd
```



Forced Admin

Administrators can force assign or override credentials for any target system user.

```
$ sudo passwd john
```



Lock Account

Locks access to the user account by appending a prefix to their hash in the shadow files.

```
$ sudo passwd -l john  
# -u to unlock
```

The **addgroup** Command

User-friendly interface tool to define new group permissions within the system's group registries.



Standard Add

Registers a standard group name using the next available higher-range GID (>1000).

```
$ sudo addgroup devops
```



Explicit GID

Creates a custom group, overriding default increment rules to secure a precise GID.

```
$ sudo addgroup --gid 2050  
devops
```



Add Membership

Appends an existing active user directly to the target group's secondary list.

```
$ sudo adduser john devops
```

The **delgroup** Command

Removes logical user group records from database tables securely, ensuring robust system management.

Direct Removal

Removes the named group record from system tables if there are no locked dependencies.

```
$ sudo delgroup devops
```

Remove Member

Removes a user from a secondary group membership without deleting either entity.

```
$ sudo deluser john devops
```

Guarded Delete

A safe delete option that halts execution if the group currently has active member users.

```
$ sudo delgroup --only-if-empty  
devops
```

05. File Permissions

Mastering discretionary access controls (DAC) on files and directories in Linux.

Linux Permission Model

Linux applies basic Discretionary Access Control (DAC) through three scopes: **Owner (u)**, **Group (g)**, and **Others (o)**.

Permission Type	Symbolic	Binary	Octal Value	Impact on Files	Impact on Directories
Read	r--	100	4	View file content (e.g. with cat)	List directory files (e.g. with ls)
Write	-w-	010	2	Modify file content or append data	Create, rename, or delete files in directory
Execute	--x	001	1	Run file as an active application	Enter directory (e.g. with cd) and access files

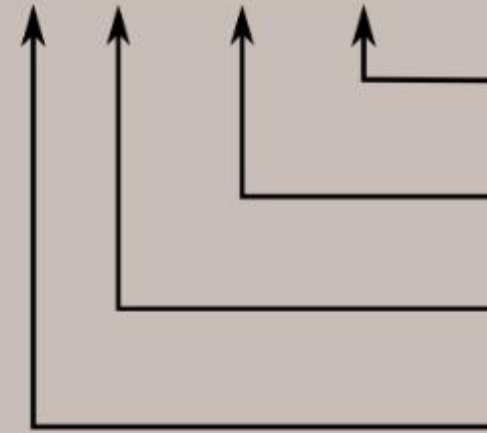
LINUX FILE PERMISSIONS

Linux secures local storage structures by assigning explicit metadata permissions across three core scopes:

- **Owner User (u):** Single user creator.
- **Group Class (g):** Assigned group context.
- **Others Scope (o):** Any external account profile.

Properties format: `[type][owner][group][others]`. For example, `drwxr-xr-x` is a directory where the owner has full read/write/execute rights.

`- rwxrwxrwx`



Read, write, and execute permissions for all other users.

Read, write, and execute permissions for the group owner of the file.

Read, write, and execute permissions for the file owner.

File type:
- indicates regular file
d indicates directory

The **chmod** Command

Adjusts security flags on folders and files using symbolic math expressions or direct numeric octal notation.



Numeric Octal

Sets exact permissions across scopes using octal values (e.g. 755: owner rwx, group/others rx).

```
$ chmod 755 run.sh
```



Symbolic Mode

Targeted adjustments using symbols (e.g., granting user/group execute permissions on a file).

```
$ chmod ug+x run.sh
```



Recursive Run

Recursively applies target permissions to all nested files and subdirectories.

```
$ chmod -R 644 /var/www
```

The **chgrp** Command

Specifically changes group ownership of files and folders without altering user-level configurations.



Change Group

Updates the primary group association of a file to the specified group.

```
$ sudo chgrp devops  
config.yml
```



Recursive Group

Recursively updates group ownership of a folder and its contents.

```
$ sudo chgrp -R devops  
configs/
```



Copy Reference

Applies group configurations cloned directly from an existing reference file.

```
$ sudo chgrp  
--reference=base.txt  
target.txt
```


The **chown** Command

Changes the owner and group ownership of directories and files, requiring superuser privileges.



Single Owner

Updates only the user owner of the target path, leaving the group owner unchanged.

```
$ sudo chown developer  
log.txt
```



Owner & Group

Simultaneously updates the user owner and the group owner of the file using a colon separator.

```
$ sudo chown ubuntu:admin  
log.txt
```



Recursive Owner

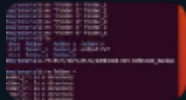
Recursively updates user and group owner permissions across an entire directory tree.

```
$ sudo chown -R www-data:www-data  
/var/www
```

Questions & Practice?

Deploy an Ubuntu Server VM instance to experiment with filesystems and practice permissions commands directly.

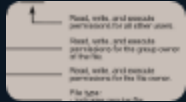
IMAGE SOURCES



<https://ubuntucommunity.s3.dualstack.us-east-2.amazonaws.com/original/2X/7/78c42837d43e6483cfff3c1b20d6d27ae89e198d.png>
Source: ubuntu.com



<https://linuxhandbook.com/content/images/2020/06/linux-directory-structure.png>
Source: linuxhandbook.com



https://miro.medium.com/v2/resize:fit:660/0*5FgkfJtRbgCQIJuk.png
Source: medium.com
